[Universidad]
[Departamento]

FUNDAMENTOS DE INFORMÁTICA

Práctica Obligatoria
Curso 2026/2027

Índice general

1. Introducción 2

1.1. Descripción general de la práctica 2

1.2. Diseño de la práctica: Diagrama de clases 3

2. ETAPA 1: Clase, objetos, atributos y métodos 5

2.1. ETAPA 1.1: Diseño 6

2.2. ETAPA 1.2 Implementación 6

2.3. ETAPA 1.3: Ejecución y pruebas 7

3. ETAPA 2: Agrupando Objetos 10

3.1. ETAPA 2.1: Diseño 10

3.2. ETAPA 2.2: Implementación 12

3.3. ETAPA 2.3: Ejecución y pruebas 12

4. ETAPA 3: Herencia 15

4.1. ETAPA 3.1: Diseño 15

4.2. Etapa 3.2: Implementación 15

4.3. Etapa 3.3: Ejecución y pruebas 16

5. ETAPA 4: Opcional 19

6. Fechas y normas de entrega 20

6.1. Fechas 20

6.2. Normas de entrega 20

6.3. Fechas de entrega 21

6.4. Evaluación de la práctica 21

6.5. Preguntas al equipo docente 21

Introducción

La práctica se realizará utilizando *Java* como lenguaje de Programación Orientado a Objetos. El primer concepto que se debe conocer es el de *objeto*, el ladrillo con el que se construye un programa bajo este paradigma de programación. Los objetos tienen vida cuando nuestro programa está en ejecución y pueden relacionarse entre sí de diversas maneras, proporcionando al programador las funcionalidades necesarias para resolver una amplia gama de problemas.

La definición de un objeto viene dada por lo que se conoce como *clase*, siendo esta la entidad que en realidad diseñaremos y programaremos en términos de sus *campos* o *atributos* y *métodos*. Estos campos y métodos nos permiten definir las características y funcionalidades de la clase. Una vez definida, a partir de la clase podremos crear objetos, que son los que realmente llevarán a cabo la funcionalidad de nuestro programa en tiempo de ejecución.

Por tanto, podemos ver los objetos como los componentes que nos servirán para diseñar nuestro programa, y las clases como las definiciones de dichos objetos. Una misma clase puede, por tanto, dar lugar a muchos objetos diferentes cada uno con características diferentes que vendrán dadas por los valores de sus campos.

1.1. Descripción general de la práctica

Esta práctica consistirá en crear un sistema informático simplificado para gestionar un gimnasio, donde los socios podrán darse de alta, registrar sus sesiones de entrenamiento y consultar la información de otros socios. Para ello, el programa realizará tres tipos de funcionalidades: el alta de socios, el registro de sesiones de entrenamiento en la ficha de cada socio y la consulta de la información de los socios dados de alta.

Este proyecto busca familiarizar al estudiante con la creación de aplicaciones informáticas, llevando a la práctica los principios de programación en Java.

En general, el proceso que se quiere informatizar es el siguiente:

1. Un socio puede darse de alta en el gimnasio y actualizar su información, que básicamente será su nombre y su ficha. La ficha incluye información sobre su fecha de nacimiento, objetivo, lesiones y nivel de actividad. El sistema le asigna a cada socio automáticamente un identificador, que consistirá en la palabra *socio* concatenada con el primer número libre. Además, el sistema le asigna una URL que tendrá el formato: `https://www.miGym.com/<identificador>`.
2. El socio puede registrar una sesión de entrenamiento, que tendrá un texto. El sistema le asignará una URL con el formato:

   ```
   https://www.miGym.com/sesion/identificador_de_socio.numero
   ```

donde el número dependerá del número de sesiones que el socio tenga registradas.

3. Se pueden visualizar los socios existentes. En concreto, se verá la lista de nombres e identificadores, o bien toda la información relativa a uno de ellos, es decir, acceder a su ficha.

Este ejercicio de programación se centrará únicamente en el *back-end*, es decir, en la parte del desarrollo encargada de la lógica necesaria para que el sistema funcione, omitiendo la interacción real con una interfaz de usuario y las comunicaciones de red; todo ello será obviado en esta práctica. En concreto, se deberá implementar un sistema capaz de realizar acciones básicas como añadir un socio, hacer un login, visualizar en pantalla los socios, añadir una sesión a la ficha de un socio, etc. La parte relativa al interfaz, como los formularios donde introducir los datos o el menú donde se eligen socios para su visualización, no forma parte de la práctica. El sistema deberá funcionar correctamente ante una serie de invocaciones a constructores y otros métodos que vendrán dados en el enunciado. Además, el material de la práctica incluye un software de interfaz que permite realizar las acciones no cubiertas por el back-end. Este software deberá poder interactuar correctamente con las clases implementadas por el alumno. De este modo, el objetivo es completar la programación de la lógica que subyace al sistema planteado, poniendo a prueba la capacidad del estudiante para estructurar y organizar el código de una manera que facilite el crecimiento iterativo y la adición de nuevas funcionalidades en futuras versiones.

Para la realización de esta práctica se seguirá un enfoque iterativo, de forma que el desarrollo se dividirá en etapas. En cada etapa se partirá del diseño y desarrollo de la etapa inmediatamente anterior, añadiéndole nuevas funcionalidades y llevando a la práctica los conceptos del temario que se van estudiando progresivamente.

Además, el enunciado de la práctica se acompaña de distintos fragmentos de código, que servirán para validar cada una de las etapas del enunciado, lo que permitirá comprobar si la solución que se está dando en cada momento es correcta o no. Ese código de comprobación estará encapsulado en distintas clases, una por cada etapa, planteando un conjunto de pruebas que permitirán validar la solución propuesta hasta ese momento. Las pruebas consistirán en llamadas a métodos de los objetos creados en cada etapa, por lo que **será muy importante respetar y seguir los identificadores que se proponen para los distintos elementos; deberán respetarse el nombre de las clases, atributos, métodos, tipos de datos, etc.** Por compatibilidad, los identificadores no incluyen caracteres especiales (por ejemplo, se utiliza *anadir* en lugar de *añadir*).

RECOMENDACIONES: La práctica se realiza a lo largo del cuatrimestre y forma parte del estudio de la asignatura. Es conveniente hacer una primera lectura completa de este documento para tener una visión global de lo que se pide, pudiéndose organizar así apropiadamente el estudio de la asignatura. Recomendamos también, especialmente en el caso de no tener experiencia previa en programación, la realización de las actividades que aparecen enlazadas como “material interactivo sobre nociones básicas” en la pestaña de “contenidos” en el curso virtual.

1.2. Diseño de la práctica: Diagrama de clases

Cuando afrontamos el desarrollo de una aplicación informática es indispensable y necesario realizar una fase previa donde se analicen los requisitos y se diseñen las entidades; en nuestro caso, las clases necesarias para llevar a cabo las funcionalidades requeridas y la relación entre las mismas. Para ello, antes de implementar cada etapa en las que se ha dividido la práctica, se deberá **realizar una lectura completa de la etapa a abordar**, realizando un diagrama donde se representen las clases necesarias con sus atributos y métodos, así como las relaciones entre las mismas, para posteriormente comenzar la fase de

implementación. Así, se deberá realizar un diagrama de clases desde la primera etapa, que luego posteriormente se irá refinando y/o modificando en cada una de las siguientes etapas, siempre como requisito previo a comenzar a implementar el código de cada etapa.

El diagrama de clases obtenido en la última etapa debe ser entregado junto al código de la práctica.

ETAPA 1: Clase, objetos, atributos y métodos

En esta primera etapa de la práctica se van a afianzar los conceptos básicos de la programación orientada a objetos: **clase**, **objeto**, **atributos** y **métodos** (constructores, métodos que ayudan a la encapsulación de la clase y métodos de visualización). El objetivo es afianzar la idea de que en programación orientada a objetos han de modelarse conceptos del problema como clases, y encapsular dentro de cada clase solo la información y procedimientos que atañen a dicho concepto.

Para ello en esta etapa, analizaremos cómo modelar una entidad básica que represente al socio en el gimnasio. Siguiendo el principio de modularización y abstracción descrito en el libro base, el problema debe descomponerse en sub-problemas, abordando cada uno de forma independiente. La clase `Socio` encapsulará la información relativa a cada socio. Esto permitirá mantener esta información aislada, facilitando la gestión de el gimnasio y garantizando la coherencia del sistema. En otras palabras, cada socio registrado en el sistema se modelará como un objeto de la clase `Socio`, con características comunes (atributos o campos). Sin embargo, los valores de estos atributos podrán ser específicos para cada socio, permitiendo representar y gestionar de manera eficiente cada socio dentro del sistema.

En el tema 6 de la asignatura (requisito previo para la realización de esta etapa de la práctica) se describen los conceptos de tipos primitivos y tipos de objeto. Es decir, a parte de tipos primitivos como `int` o `boolean`, se puede definir como atributo de una clase otra clase (ver el capítulo 3 del libro base). Esta relación entre objetos es lo que se denomina **composición**. Esta relación permite la modularización del problema.

En nuestro escenario, por ejemplo, la ficha de un socio contiene a su vez sus propios campos, como fecha de nacimiento, objetivo etc. Por tanto, siguiendo la metodología de orientación a objetos, debemos encapsular la ficha como una clase independiente y relacionarla con el socio mediante composición. De esta forma, trabajamos las nociones de *abstracción* y *modularización* descritas en el tema 6 de la asignatura (Interacción entre objetos) y más concretamente en la sección 3.2 del libro base.

Además, en esta etapa deberán ponerse en práctica los principios de encapsulación de la programación orientada a objetos tal y como se describe en el libro base de la asignatura. Esto implica que los campos de las clases deben estar definidos como privados, siendo los métodos de la propia clase los únicos que tienen acceso a dichos campos.

Para la realización de esta primera etapa se requiere haber estudiado los temas 4, 5 y 6 del temario detallado de la asignatura, correspondientes a los capítulos 1, 2 y 3, así como los apéndices A, B, C, D y F del libro de referencia de la Unidad Didáctica II. Para la realización de la etapa 1.2, deben consultarse capítulos 6.15 y 6.16 del libro de texto, sobre métodos de clase. Además, en el apéndice E puede encontrar más información

sobre el método `main()`, en concreto en el apartado E1. Puede consultar también el capítulo 3.10 “Objetos que crean objetos” si necesita más información acerca de la creación de objetos desde el código.

2.1. ETAPA 1.1: Diseño

Uno de los elementos principales del sistema es la noción de socio, que será modelada como una clase a partir de la cual se podrán crear instancias u objetos de la misma. En esta fase del desarrollo de la práctica, cada socio se describe por la siguiente información, que modelaremos como atributos de la clase `Socio`:

- `nombre`, que representa el nombre y apellidos real del socio.
- `idSocio`, que representa el identificador dado por el sistema, que tendrá el formato `socioX`, donde X es un número.
- `password`, que representa la clave de seguridad para poder hacer un login.
- `URL`, que representa la dirección URL asignada por el sistema al socio, que tendrá el formato: `https://www.miGym.com/identificador`.

Además, cada socio dispone de información en su ficha. Esto incluye fecha de nacimiento, y tres textos correspondientes a su objetivo, sus lesiones y su nivel de actividad.

El sistema debe permitir instanciar objetos de la clase `Socio` y `FichaSocio`, así como actualizar la ficha del socio mediante el método set correspondiente. Además, la clase `Socio` dispondrá de un método que imprima en pantalla toda su información, incluyendo nombre, URL, ficha, etc. Se muestra a continuación un ejemplo de formato que debería tener la salida en pantalla de este método.

```
Identificador de socio: socio1
     Nombre: Laura Prieto Gil
     URL: https://www.miGym.com/socio1
     FICHA:
          Fecha de nacimiento: Sun Jan 24 00:00:00 CET 1988
          Objetivo: Ganar masa muscular
          Lesiones: Ninguna
          Nivel de actividad: Intermedio
```

Siguiendo la metodología de programación orientada a objetos, es importante tener en cuenta que cada clase debe incluir los métodos que gestionan sus propios atributos. Esto quiere decir que la impresión de los atributos del socio y de la ficha debe realizarse en métodos de su clase correspondiente. Téngase en cuenta que el método que imprime la información del socio puede invocar a su vez al método que imprime la información de la ficha.

La tarea en esta etapa consistirá en realizar un diseño con las clases `Socio` y `FichaSocio` relacionadas mediante composición. El diseño debe incluir los atributos y métodos de cada una de las clases. Por cada atributo debe haber dos métodos, uno para consultar su valor y otro para modificarlo. Por ejemplo, para el campo `nombre` tendremos los métodos `getNombre()` y `setNombre()` para consultar su valor y modificarlo, respectivamente.

2.2. ETAPA 1.2 Implementación

La tarea correspondiente a esta etapa consiste en programar el diseño realizado en la etapa anterior. Para comprobar su funcionamiento se puede usar el entorno BlueJ.

NOTA SOBRE BLUEJ: Es importante comprender que *Java* y *BlueJ* son cosas diferentes. Java es un lenguaje de programación, mientras que BlueJ es un entorno de programación que nos permite programar en Java. BlueJ está pensado para el aprendizaje de Java y proporciona distintas herramientas que permiten inspeccionar las partes de un programa, tanto en la parte de diseño (cuando estamos escribiendo el código) como en la de ejecución (cuando el código se ejecuta, se crean los objetos en memoria, etc.).
Una de las herramientas de BlueJ que pueden resultar más útiles es el banco de objetos u *object bench* (capítulo 1 del libro), que junto con otras herramientas como el *inspector de objetos*, resulta muy interesante para "jugar" con los objetos en tiempo de ejecución de forma interactiva.
BlueJ nos permite ejecutar nuestras aplicaciones mediante la invocación de los métodos de sus objetos, pero no es esta la única manera de hacerlo. Java, más allá de BlueJ, nos permite ejecutar los programas que creamos de forma independiente. A estas alturas sabemos que cada clase tiene unos métodos y campos propios de los objetos creados a partir de dicha clase, que para poderlos usar necesitamos crear primero un objeto. Por ejemplo, la clase `Socio` que hemos creado tiene un método para consultar su identificador, que sólo podría ser invocado una vez tengamos creado algún objeto de esta clase.
Al final de este apartado se debería entender la diferencia entre ejecutar un programa en BlueJ y hacerlo de forma independiente, esto es, por ejemplo, ejecutarlo desde la línea de comandos (símbolo de sistema en Windows/Linux/MacOS).

Para la fase de implementación damos las siguientes guías sobre el uso de clases predefinidas. En cuanto a la fecha de nacimiento, podemos usar el tipo `Date`. En este sentido establecemos una relación de composición sobre una clase que no tendremos que programar, sino que podremos importarla directamente incorporando una librería. Para ello debemos añadir en nuestro código la instrucción `import java.util.Date;`. Más adelante en este curso, veremos más detalles sobre el uso de librerías en la sección 5.1 del libro base. Por el momento, basta considerar que definir un atributo como de tipo `Date` implica establecer una relación entre la clase `FichaSocio` y la clase `Date` importada desde una librería.

Para los otros atributos de la clase ficha, así como los de la clase que representa al socio, se puede usar el tipo `String`. Esto quiere decir que, al igual que en el caso de la fecha de nacimiento, también se trata de una relación de composición entre objetos. La diferencia es que en este caso la clase "viene implementada por defecto", o lo que es lo mismo, pertenece al paquete `java.lang` que se importa automáticamente sin necesidad de especificarlo en el código.

En la etapa 2 (siguiente etapa) implementaremos los métodos que asignan automáticamente identificadores y URL a los socios. Para mantener la encapsulación, estas acciones serán llevadas a cabo por otras clases que tienen acceso a todos los socios. Es decir, en programación orientada a objetos es muy importante que cada clase realice solo aquellas acciones que pueden ser llevadas a cabo accediendo a la información que encapsula.

2.3. ETAPA 1.3: Ejecución y pruebas

Tener una clase implementada no es suficiente para que un programa se ejecute porque una clase, en realidad, no es un programa en sí, sino una definición abstracta de cómo deben ser ciertos objetos y qué comportamiento pueden tener. La clase establece atributos, métodos y relaciones, pero no indica cuándo debe comenzar la ejecución ni qué acciones concretas deben realizarse primero. Por ejemplo, en nuestro caso, la clase que hemos programado define las características de un socio, qué información incluye y qué acciones (métodos) se pueden realizar sobre dicha información.

Los métodos definidos en una clase no se ejecutan automáticamente; pertenecen a instancias concretas de esa clase. Por eso es necesario crear objetos, es decir, instanciar la clase, lo que implica reservar memoria y dar lugar a un elemento concreto con estado propio. Solo una vez creada la instancia es posible utilizar sus métodos y acceder a sus atributos. En ausencia de un punto de entrada, de instancias y de llamadas explícitas a los métodos, el código queda limitado a ser una colección de definiciones estáticas, correctas

desde el punto de vista sintáctico, pero incapaces de producir ninguna acción o resultado durante la ejecución.

Para ello, se debe implementar una clase con un método estático (ejecutable) que en JAVA se denomina `main()`. A partir del método `main()`, se crean estas instancias y se realizan las invocaciones necesarias para que el sistema se pueda ejecutar realizando acciones en un orden determinado.

En esta práctica comprobaremos de dos maneras el correcto funcionamiento de las clases que implementemos. Por un lado, ejecutando una clase con un método `main` que contiene código donde se instancian las clases y se llaman a métodos que aportamos directamente en el enunciado. Estas clases ejecutables se llamarán `EjemploEtapaX`, donde X representa la etapa en la que nos encontremos dentro del desarrollo de la práctica. Por otro lado, simulando un escenario más realista, se aporta junto con el material de esta práctica una serie de clases denominadas `InterfazEtapaX` en donde se ha programado un interfaz que hace uso de las clases que implementemos a lo largo de la práctica.

El objetivo de esta etapa es ejecutar estas clases para comprobar que la clase `Socio` se haya implementado correctamente. Para ejecutar nuestro programa de manera independiente de BlueJ y crear el flujo principal del sistema, crearemos una clase llamada `EjemploEtapa1`, donde implementaremos el método `main()`. Esta clase contendrá el código en el que se crean las instancias de objetos necesarias y las llamadas para poder comprobar el funcionamiento del sistema.

El ejercicio a realizar en este punto consiste en crear dicha clase `EjemploEtapa1`, que contenga un método `main()` e implementar en este método el siguiente código[1]:

```
import java.util.Date;

public class EjemploEtapa1 {
    public static void main (String [ ] args) {
        Socio p=new Socio();
        p.setIdSocio("socio1");
        p.setNombre("Laura Prieto Gil");
        p.setURL("https://www.miGym.com/socio1");
        p.setPassword("pesas2024");
        FichaSocio f=new FichaSocio();
        Date fecha = new Date(88, 0, 24);
        f.setFechaDeNacimiento(fecha);
        f.setObjetivo("Ganar masa muscular");
        f.setLesiones("Ninguna");
        f.setNivelActividad("Intermedio");
        p.setFicha(f);
        p.imprimeInformacion();
    }
}
```

Este código debería crear una instancia de la clase `Socio` y otra de la clase `FichaSocio` y fijar sus atributos. Posteriormente, el código imprime por pantalla la información siguiente:

```
Identificador de socio: socio1
     Nombre: Laura Prieto Gil
     URL: https://www.miGym.com/socio1
     FICHA:
          Fecha de nacimiento: Sun Jan 24 00:00:00 CET 1988
          Objetivo: Ganar masa muscular
          Lesiones: Ninguna
          Nivel de actividad: Intermedio
```

Además, deberá poder ejecutarse correctamente la clase `InterfazEtapa1` que se proporciona junto a este enunciado. Al ejecutar este programa, se pide por línea de comandos el identificador del socio, su nombre, ficha y URL. Luego se piden los datos de su ficha. El programa crea los objetos correspondientes, e imprime la información. Este programa realiza estas operaciones mediante llamadas a constructores y métodos de la práctica implementada. Es importante que la práctica siga las especificaciones y cubra los

ejemplos de instanciación y llamadas a métodos.

NOTA IMPORTANTE: La presente práctica debe poderse ejecutar independientemente de BlueJ de cara a su entrega y corrección.

[1] En el curso virtual, junto con el enunciado de la práctica, puede descargar el código fuente de esta clase EjemploEtapa1

ETAPA 2: Agrupando Objetos

Esta etapa se corresponde con los temas 7, 8 y 9 de la asignatura. Además de implementar nuevas clases y relaciones de composición, se abordará el problema de la agrupación de objetos en estructuras y la implementación de bucles para el procesamiento de dichas estructuras.

En esta etapa deberán también ponerse en práctica los principios de encapsulación de la programación orientada a objetos tal y como se describe en el libro base de la asignatura (sección 5.11 del libro base). Esto implica que los campos de las clases deben estar definidos como privados, siendo los métodos de la propia clase los únicos que tienen acceso a dichos campos.

Finalmente, es una buena práctica de programación documentar todo el código que se va realizando, no sólo con el objetivo de que terceras personas puedan comprender nuestro programa, sino también para que nosotros mismos podamos entender nuestro código de cara a futuras extensiones, mejoras o modificaciones. Para la generación de documentación, Java nos ofrece la herramienta **Javadoc** (sección 5.10 del libro base), que por medio de determinado tipo de comentarios añadidos en nuestro código fuente, nos permite la generación posterior de toda la documentación de las distintas clases.

Para la realización de esta etapa se deben estudiar los temas 7, 8 y 9 del temario detallado de la asignatura. Encontrará información más detallada sobre el mecanismo de composición y la generación de documentación en el libro de texto de la asignatura en los capítulos 4, 5 y 6. La lectura de las secciones de 8.3 a 8.6 (ambas inclusive) sobre buenas prácticas en el diseño de aplicaciones es también recomendable.

3.1. ETAPA 2.1: Diseño

En esta etapa ampliaremos el diseño incorporando la noción de sesión, incorporando una nueva clase a la que llamaremos `Sesion`. Siguiendo las especificaciones indicadas al comienzo de este enunciado, una sesión se caracteriza por un texto (contenido de la sesión) y una URL con el formato `https://www.miGym.com/sesion/identificador_de_socio.numero`

En la etapa anterior abordamos la relación de composición, estableciendo una relación uno a uno entre socios y fichas. De este modo, encapsulábamos la información relativa a las fichas en una clase independiente. Sin embargo, la mayoría de las relaciones entre clases son de uno a muchos. Por ejemplo, un socio tiene asociado un conjunto de sesiones. Se trata de relaciones de composición dado que las sesiones forman parte del socio como uno de sus atributos.

Para capturar relaciones de uno a muchos es necesario introducir una estructura intermedia que agrupe todas las sesiones como un único atributo del socio. Para ello utilizaremos la clase `ArrayList`. En la sección 4.4 del libro se explican con más detalle sus características y funcionamiento. Esta clase permite almacenar y gestionar listas de elementos de forma sencilla y flexible, ya que posibilita añadir y eliminar elementos sin necesidad de fijar el tamaño de la colección desde el inicio. Los elementos se almacenan en

un orden determinado y pueden accederse mediante una posición numérica, de manera similar a un array, comenzando por la posición cero. Por estas razones, `ArrayList` es una estructura muy utilizada cuando se trabaja con colecciones de datos de tamaño variable de forma cómoda y eficiente. De esta forma, la clase `Socio` tendrá como atributo un `ArrayList` que a su vez contiene objetos del tipo `Sesion`.

En este nuevo diseño la impresión en pantalla de la información de un socio debe incluir la información de todas sus sesiones. Además, el diseño debe permitir añadir una sesión al socio. En el momento en que se añade, automáticamente se le asigna una URL formato:

```
https://www.miGym.com/sesion/identificador_de_socio.numero
```

Es importante tener en cuenta que es la clase `Socio` la que debe asignar este identificador cuando se añada una sesión al socio, dado que es ésta clase la que tiene control sobre las sesiones del socio.

En esta etapa diseñaremos también la clase `MiGym` que va a modelar el gimnasio y que gestiona todos los socios, además de ofrecer funcionalidades mediante una serie de métodos que describiremos a continuación:

- Dar de alta un socio en el sistema. Para ello debe disponer de un método `anadirSocio` que reciba como entrada un objeto de tipo `Socio`. El sistema le asignará un identificador de socio que consistirá en la palabra *socio* y el primer número libre. Además, le asignará una URL con el formato:

  ```
  https://www.miGym.com/<identificador_de_socio>
  ```
- Un método `comprobarPassword` que, dado un identificador de socio y una password, compruebe que ésta es correcta. En caso de que no exista el identificador de socio o no se corresponda la password, el método deberá indicarlo imprimiendo en pantalla el resultado y devolviendo el valor `false`. En el caso de que la password sea correcta, deberá indicarlo en pantalla y devolver el valor `true`. Es decir, en este caso el método `comprobarPassword` deberá devolver un tipo booleano.
- Visualizar los nombres e identificadores de socios dados de alta en el sistema mediante un método al que llamaremos `visualizarGimnasio`. Este método deberá imprimir en pantalla sucesivamente los identificadores de socio y su nombre para todos los socios dados de alta en el sistema.
- Un método `visualizarSocio` que dado un identificador de socio, muestre toda la información de su ficha. Para ello, el método tendrá que comprobar que existe el socio, y en caso contrario indicarlo en pantalla.
- Un método `anadirSesion`, que reciba como entrada el identificador de socio y el texto de la sesión. El sistema deberá comprobar que el socio existe antes de añadir la sesión a su ficha. En caso de que no exista, deberá escribir un mensaje en pantalla indicando que el socio no existe.

En este punto es importante recordar los conceptos de *acoplamiento* y *cohesión* descritos en el capítulo 6 del libro base. Los métodos `comprobarPassword`, `visualizarSocio` y `anadirSesion` deben comprobar que existe el socio. La forma correcta de diseñar una solución para esto sería añadir un método auxiliar `buscarSocio` que devuelva `null` en el caso de que el socio no exista o el objeto del tipo `Socio` correspondiente en el caso de que sí exista. Esto permitiría que si por algún motivo queremos optimizar esta comprobación, por ejemplo mediante algún algoritmo de búsqueda más eficiente, podamos hacerlo una sola vez y que no sea necesario programar lo mismo en varias partes del código.

También es necesario recordar el concepto de *reutilización de código* en programación orientada a objetos. Al igual que nosotros hemos reutilizado el código de clases como `ArrayList` o `Date` que tienen sus propios métodos, los desarrolladores de la aplicación final usarán nuestro código sobre el que implementarán un sistema que incluya otros aspectos como el interfaz de usuario, almacenamiento de información, login, seguridad, etc. Estos desarrolladores necesitarán de otras funcionalidades que tenemos que implementar. Por ejemplo, el método `comprobarPassword` será usado por el sistema global para

permitir que los socios accedan a la aplicación.

La tarea a realizar en esta etapa consiste en hacer el diseño de clases cubriendo las especificaciones anteriores.

3.2. ETAPA 2.2: Implementación

En esta etapa implementaremos el diseño desarrollado en la etapa anterior. Para ello damos las siguientes directrices que pueden ayudar en el proceso de implementación.

Hay que recordar que la clase `ArrayList` requiere importar una librería: `import java.util.ArrayList;` Es decir, al definir el atributo `sesiones` como una colección del tipo `ArrayList` con objetos de la clase `Sesion`, esto nos permite hacer uso de métodos ya implementados en otras librerías. En otras palabras, estamos reutilizando código. Lo mismo para el problema de composición entre `MiGym` y `Socio`. En la sección 4.4 del libro base se dan detalles sobre el uso de esta clase.

Al añadir una sesión a un socio, según las especificaciones es necesario asignarle una URL que termina con el identificador de socio y un número. Para que cada sesión tenga su propia URL, este número debe corresponderse con la cantidad de sesiones que haya en ese momento en la ficha. Para ello puede hacerse uso del método `size()` de la clase `ArrayList`. Lo mismo ocurrirá al asignarle un identificador de socio a un nuevo socio, por ejemplo *socio12*, donde el número represente la cantidad de socios en el momento de añadir el nuevo socio.

En este proceso de implementación, tendremos que modificar el método de impresión de la clase `Socio`, de forma que, siempre que existan, se impriman en pantalla las sesiones. Para comprobar que existen sesiones se puede comprobar la condición `sesiones != null`, donde `sesiones` es un `ArrayList`. Lo que estaríamos haciendo es comprobar que el atributo `sesiones` no ha sido inicializado con ninguna sesión.

Según las especificaciones, los métodos `comprobarPassword`, `visualizarSocio` y `anadirSesion` deben comprobar que existe el socio. Para ello tendrán que recorrer la lista de socios dados de alta en el sistema, encontrar aquel que coincida en el identificador. Podemos reutilizar el método `equals` de la clase `String` que comprueba si dos string son iguales.

3.3. ETAPA 2.3: Ejecución y pruebas

Una vez implementada la clase `MiGym`, debería poder ejecutarse el siguiente código en el método `main()`, que puede encontrarse en el fichero `EjemploEtapa2`. A continuación, desglosamos por pasos el contenido de este fichero y la respuesta que debería de aparecer en pantalla.

```
import java.util.Date;

public class EjemploEtapa2 {
    public static void main (String [ ] args) {
        Socio p1=new Socio("Laura Prieto Gil","pesas2024");
        FichaSocio f=new FichaSocio();
        Date fecha = new Date(88, 0, 24);
        f.setFechaDeNacimiento(fecha);
        f.setObjetivo("Ganar masa muscular");
        f.setLesiones("Ninguna");
        f.setNivelActividad("Intermedio");
        p1.setFicha(f);
        Socio p2=new Socio("Álvaro Muñoz Peña","cardio99");
        FichaSocio f2=new FichaSocio();
        Date fecha2 = new Date(95, 1, 9);
        f2.setFechaDeNacimiento(fecha2);
        f2.setObjetivo("Perder peso");
        f2.setLesiones("Rodilla derecha");
```

```
        f2.setNivelActividad("Principiante");
        p2.setFicha(f2);
        MiGym r=new MiGym();
        r.anadirSocio(p1);
        r.anadirSocio(p2);
        r.visualizarGimnasio();
        ...
```

Hasta este punto, el código de `EjemploEtapa2` debería crear dos instancias de `Socio` con toda su información menos la URL y el identificador de socio. Entonces, al realizar las llamadas a `anadirSocio` el sistema debería asignar los identificadores de socio y las URL. Tras esto, al visualizar el gimnasio, debería verse en pantalla la lista con los dos identificadores de socio y sus nombres.

```
Socios en el gimnasio:
     socio1: Laura Prieto Gil
     socio2: Álvaro Muñoz Peña
```

A continuación, mediante el método `visualizarSocio` debería desplegarse la información de cada uno de los socios. Es decir, lo siguiente en el código de `EjemploEtapa2` será:

```
        ...
        r.visualizarSocio("socio56");
        r.visualizarSocio("socio1");
        ...
```

En el primer caso el sistema deberá indicar que no existe el socio. Ante la siguiente instrucción el sistema deberá mostrar la información del socio:

```
No existe el socio socio56
Identificador de socio: socio1
     Nombre: Laura Prieto Gil
     URL: https://www.miGym.com/socio1
     FICHA:
          Fecha de nacimiento: Sun Jan 24 00:00:00 CET 1988
          Objetivo: Ganar masa muscular
          Lesiones: Ninguna
          Nivel de actividad: Intermedio
```

A continuación, `EjemploEtapa2` hace uso del método `comprobarPassword`:

```
        ...
        boolean comprobacion1 = r.comprobarPassword("socio56","pesas");
        boolean comprobacion2 = r.comprobarPassword("socio1","pesas");
        boolean comprobacion3 = r.comprobarPassword("socio1","pesas2024");
        ...
```

Entonces, el sistema debería imprimir en pantalla los mensajes correspondientes a cada llamada:

```
No existe el socio socio56
La password es incorrecta
Password correcta
```

A continuación, el código de `EjemploEtapa2` hace uso del método `anadirSesion`. Pero emplea un identificador de socio inexistente. El sistema debe de informar de ello. Tras esto llama a `anadirSesion`, y visualiza la ficha.

```
        ...
        r.anadirSesion("socio56","Entrenamiento de fuerza: pecho y espalda");
        r.anadirSesion("socio1","Entrenamiento de fuerza: pecho y espalda");
        r.anadirSesion("socio1","Clase de spinning de 45 minutos");
        r.visualizarSocio("socio1");
    }
}
```

A esta parte de código, el sistema debería de responder:

```
No existe el identificador de socio socio56
Identificador de socio: socio1
     Nombre: Laura Prieto Gil
     URL: https://www.miGym.com/socio1
     FICHA:
          Fecha de nacimiento: Sun Jan 24 00:00:00 CET 1988
          Objetivo: Ganar masa muscular
          Lesiones: Ninguna
          Nivel de actividad: Intermedio
     SESIONES:
          https://www.miGym.com/sesion/socio1.1 Entrenamiento de fuerza: pecho y espalda
          https://www.miGym.com/sesion/socio1.2 Clase de spinning de 45 minutos
```

Además, deberá poder ejecutarse correctamente la clase `InterfazEtapa2` que se proporciona junto a este enunciado. Al ejecutar este programa, se pide por línea de comandos el nombre de dos socios y sus datos. El interfaz crea las fichas correspondientes y las añade al sistema. Tras esto, el interfaz ofrece tres opciones: visualizar socio, añadir sesión o salir. En el primer caso, muestra en pantalla los identificadores de socio y sus nombres. Luego pregunta por un identificador de socio y muestra la información de su ficha. En el segundo caso también muestra en pantalla los identificadores de socio y sus nombres. Luego pregunta por un identificador de socio, la password y el texto de la sesión a añadir. Si el socio existe y la password es correcta, añade la sesión e imprime en pantalla la información de la ficha.

ETAPA 3: Herencia

Esta etapa aborda los contenidos de los temas 10, 11 y 12 del temario detallado de la asignatura, correspondientes a los Capítulos 10 y 11, secciones 12.3 y 12.4 y el capítulo 14 y 7 del libro base para la Unidad Didáctica II. Vamos a analizar un aspecto fundamental de la programación orientada a objetos, la *herencia*. A lo largo del ciclo de vida de un sistema informático pueden surgir nuevos requisitos, es decir, la necesidad de cubrir nuevas funcionalidades. Antes de que se popularizara este paradigma de programación, era necesario replantear el sistema y revisar el código ya implementado. Sin embargo, la programación orientada a objetos ofrece mecanismos de *herencia*, de forma que, a partir del código ya existente, pueden definirse nuevas clases que *heredan* las propiedades de las clases ya existentes pero incorporando nuevos atributos o métodos.

4.1. ETAPA 3.1: Diseño

En esta parte de la práctica vamos a simular la situación en la que se quiere extender el sistema para que se contemplen socios *premium*. La particularidad de los socios premium es que contienen, además del nombre, URL y ficha, un entrenador personal asignado y un campo con la descripción de su plan nutricional.

En cuanto a diseño, es importante tener en cuenta algunos aspectos. El uso de la herencia debería permitir extender el sistema sin modificar la arquitectura previa ya implementada. Debería bastar con crear una clase hija a partir de las existentes. En el diseño debe indicar los nuevos atributos o campos de la clase hija, así como los métodos que es necesario reescribir. Recordemos que la reescritura de métodos en la herencia consiste en que si en la clase hija aparece un método con la misma cabecera (nombre y parámetros) que un método de la clase padre, entonces cuando se invoque al objeto no se ejecutará el método de la clase padre, sino el nuevo método implementado en la clase hija.

La tarea a realizar en esta etapa consiste en hacer el diseño de clases cubriendo las especificaciones anteriores.

4.2. Etapa 3.2: Implementación

Para la implementación en esta etapa debemos de tener en cuenta algunos aspectos. Para que se pueda validar la práctica sobre las llamadas descritas en la sección 4.3, tendremos que llamar a la nueva clase hija `SocioPremium`. Al igual que su clase padre, esta clase tendrá dos parámetros en el constructor. Por otro lado, los campos propios de la clase hija podrán actualizarse con dos métodos que se denominan `setEntrenador` y `setPlanNutricional` y que reciban como entrada un objeto tipo `String` con el contenido de cada uno de los campos.

También a nivel de implementación, es importante tener en cuenta que la clase `MiGym` debería poder gestionar objetos de tipo `Socio` o `SocioPremium` indistintamente. Es lo que se denomina polimorfismo.

Si es necesario, por ejemplo asignarle una URL o identificador, `MiGym` debería poder hacer las mismas llamadas a métodos independientemente de que se trate de una instancia de la clase `Socio` o `SocioPremium`. Muchos de los métodos que se ejecutarán al llamar a `SocioPremium` serán en realidad los ya implementados en `Socio` dado que estos se heredan. Pero sin embargo, si necesitamos modificar un método como el responsable de imprimir la información, entonces la cabecera del método en la clase hija tendrá que ser la misma que usamos en la clase padre. Automáticamente se ejecutará el método original o el re-escrito dependiendo de que se trate de un objeto de la clase `Socio` o `SocioPremium`.

Otro detalle a considerar es que en la clase `MiGym` los objetos de la clase `Socio` que gestiona están definidos como `Socio`. Sin embargo, los objetos pueden pertenecer a la clase `Socio` o `SocioPremium`. Esto no supone ningún problema. Un objeto de la clase hija puede estar definido como un objeto de la clase padre. Hay que tener en cuenta que esto solo sucede en el caso de la relación de herencia. Por ejemplo, no podemos asignar a una variable un objeto del tipo `SocioPremium` si esta variable está definida como del tipo `MiGym` o `FichaSocio` dado que mantienen relaciones de composición. Sin embargo, sí que podemos asignar a una variable un objeto del tipo `SocioPremium` si esta variable está definida como del tipo `Socio`.

4.3. Etapa 3.3: Ejecución y pruebas

Una vez implementada la clase `SocioPremium`, debería poder ejecutarse el siguiente código en el método `main()`, que puede encontrarse en el fichero `EjemploEtapa3`. A continuación, desglosamos por pasos el contenido de este fichero y la respuesta que debería de aparecer en pantalla.

```
import java.util.Date;

public class EjemploEtapa3 {
    public static void main (String [ ] args) {
        Socio p1=new Socio("Laura Prieto Gil","pesas2024");
        FichaSocio f=new FichaSocio();
        Date fecha = new Date(88, 0, 24);
        f.setFechaDeNacimiento(fecha);
        f.setObjetivo("Ganar masa muscular");
        f.setLesiones("Ninguna");
        f.setNivelActividad("Intermedio");
        p1.setFicha(f);
        SocioPremium p2=new SocioPremium("Álvaro Muñoz Peña","cardio99");
        FichaSocio f2=new FichaSocio();
        Date fecha2 = new Date(95, 1, 9);
        f2.setFechaDeNacimiento(fecha2);
        f2.setObjetivo("Perder peso");
        f2.setLesiones("Rodilla derecha");
        f2.setNivelActividad("Principiante");
        p2.setFicha(f2);
        p2.setEntrenador("Marcos Vidal Ortiz");
        p2.setPlanNutricional("Dieta de definición de 2200 calorías");
        MiGym r=new MiGym();
        r.anadirSocio(p1);
        r.anadirSocio(p2);
        r.visualizarGimnasio();
        ...
```

Hasta este punto, el código de `EjemploEtapa3` debería crear dos instancias de `Socio` y `SocioPremium` con toda su información menos la URL y el identificador de socio. Entonces, al realizar las llamadas a `anadirSocio` el sistema debería asignar los identificadores de socio y URL. Tras esto, al visualizar el gimnasio, debería verse en pantalla la lista con los dos identificadores de socio y sus nombres.

```
Socios en el gimnasio:
     socio1: Laura Prieto Gil
     socio2: Álvaro Muñoz Peña
```

A continuación, mediante el método `visualizarSocio` debería desplegarse la información de cada uno de los socios. Es decir, lo siguiente en el código de `EjemploEtapa3` será:

```
        ...
        r.visualizarSocio("socio2");
        ...
```

El sistema deberá mostrar la información del socio, incluyendo los campos propios de la clase hija:

```
Identificador de socio: socio2
     Nombre: Álvaro Muñoz Peña
     URL: https://www.miGym.com/socio2
     FICHA:
          Fecha de nacimiento: Thu Feb 09 00:00:00 CET 1995
          Objetivo: Perder peso
          Lesiones: Rodilla derecha
          Nivel de actividad: Principiante
     Entrenador personal: Marcos Vidal Ortiz
     Plan nutricional: Dieta de definición de 2200 calorías
```

A continuación `EjemploEtapa3` hace uso del método `comprobarPassword`:

```
        ...
        boolean comprobacion1 = r.comprobarPassword("socio56","pesas");
        boolean comprobacion2 = r.comprobarPassword("socio2","card");
        boolean comprobacion3 = r.comprobarPassword("socio2","cardio99");
        ...
```

Entonces, el sistema debería imprimir en pantalla los mensajes correspondientes a cada llamada:

```
No existe el socio socio56
La password es incorrecta
Password correcta
```

A continuación, el código de `EjemploEtapa3` hace uso del método `anadirSesion`, y visualiza la ficha.

```
        ...
        r.anadirSesion("socio2","Rutina de piernas y glúteos");
        r.anadirSesion("socio2","Sesión de natación suave");
        r.visualizarSocio("socio2");
    }
}
```

A esta parte de código, el sistema debería de responder:

```
Identificador de socio: socio2
     Nombre: Álvaro Muñoz Peña
     URL: https://www.miGym.com/socio2
     FICHA:
          Fecha de nacimiento: Thu Feb 09 00:00:00 CET 1995
          Objetivo: Perder peso
          Lesiones: Rodilla derecha
          Nivel de actividad: Principiante
     SESIONES:
          https://www.miGym.com/sesion/socio2.1 Rutina de piernas y glúteos
          https://www.miGym.com/sesion/socio2.2 Sesión de natación suave
     Entrenador personal: Marcos Vidal Ortiz
     Plan nutricional: Dieta de definición de 2200 calorías
```

Además, deberá poder ejecutarse correctamente la clase `InterfazEtapa3` que se proporciona junto a este enunciado. Al ejecutar este programa, se dan tres opciones, “añadir socio”, “añadir sesión” y “visualizar socio”. En el primer caso se pide toda la información de la ficha: nombre, año, mes y día de nacimiento y los datos de su ficha. Luego se pregunta si es un socio *premium*. Si lo es, se piden los datos adicionales correspondientes.

En el caso de elegir “Añadir sesión”, se muestra en pantalla los identificadores de socio y sus nombres. Luego pregunta por un identificador de socio, la password y el texto de la sesión a añadir. Si el socio existe y la password es correcta, añade la sesión e imprime en pantalla la información de la ficha.

En el caso de la tercera opción, se muestra en pantalla los identificadores de socio y sus nombres. Luego pregunta por un identificador de socio y muestra la información de su ficha.

ETAPA 4: Opcional

En esta cuarta etapa opcional se debe ampliar las funcionalidades del sistema siguiendo los mismos tres pasos que en las etapas anteriores, es decir, una fase de diseño donde se presente la arquitectura del sistema mediante un diagrama de clases, la implementación del sistema cubriendo las nuevas funcionalidades, y códigos de ejemplo y respuestas obtenidas (ejecución y pruebas).

En este caso el código de pruebas debe ser generado directamente por el alumno. Es importante describir qué hace el código y la respuesta que se obtiene, además de justificar las modificaciones en el diseño e implementación.

Realizando esta parte opcional se podrá obtener un sobresaliente.

Las nuevas funcionalidades a cubrir serán:

1. **Eliminación de socios.** La clase `MiGym` debe incluir un método que dado un identificador de socio, compruebe que existe y lo elimine. Esta funcionalidad presenta complicaciones dado que hasta ahora bastaba con asignar como identificador de cada socio un número que se corresponde con el tamaño de la lista de socios existentes. Esto ya no es así. Por tanto, a la hora de asignar identificadores, deberá no solo considerarse el número de socios actuales sino el primer número que quede libre.
2. **Eliminación de sesiones.** La clase `MiGym` debe incluir un método que dado un identificador de socio y una URL de sesión, elimine de él la sesión correspondiente. En este caso, sucederá lo mismo que en el caso anterior, dado que la asignación de las URL ahora tendrá que tener en cuenta no el número de sesiones del socio, sino el primer número entero liberado.

Fechas y normas de entrega

6.1. Fechas

La realización de la práctica se llevará a cabo en los Centros Asociados, siendo las sesiones organizadas y supervisadas por el tutor de la asignatura. Habrá como mínimo tres sesiones presenciales de obligatoria asistencia. **Los estudiantes deberán ponerse en contacto con su centro asociado para informarse acerca de cuándo tendrán que asistir a las sesiones.**

Las fechas orientativas para la realización de cada una de las etapas serán:

- Finales de marzo. Realización de la primera etapa.
- Mediados de abril. Realización de la segunda etapa.
- Principios de mayo. Realización de la tercera etapa.

6.2. Normas de entrega

La práctica se entregará a través de una tarea definida en el entorno virtual de la asignatura. Cada estudiante creará y comprimirá una carpeta nombrada con su DNI y primer apellido separados por un guion ("12345678X-gonzalez"). Esta carpeta contendrá:

- Una memoria de **no más de 7 páginas**. Por cada etapa una página dedicada a la descripción del diseño (diagrama) y otra página dedicada a la descripción de las pruebas realizadas y resultados obtenidos, más una página extra en caso de que se realice la parte voluntaria.
- Los ficheros *.java*, sin caracteres especiales; por ejemplo "ñ" o tildes finales, es decir, únicamente los de la última etapa.

NOTA IMPORTANTE: Los nombres de los ficheros y carpetas/paquetes que compongan la práctica entregada deben contener SÓLO caracteres correspondientes a las letras de la A a la Z, tanto mayúsculas como minúsculas, números del 0 al 9 y los caracteres especiales '-' y '_'. **No deben utilizarse otros, tales como tildes o símbolos.**

NOTA IMPORTANTE: Los tutores tienen que cumplir una serie de requisitos ante los estudiantes debido a que la práctica cuenta para la calificación de la asignatura. Por tanto, antes de entregar las calificaciones definitivas al equipo docente deberán:
1. Informar de la nota de las prácticas a los estudiantes.
2. Establecer un día de revisión de prácticas (previo al período de entrega de las calificaciones al equipo docente), dado que éstas forman parte de la evaluación del estudiante.

Es importante que se mantengan todos los identificadores definidos en el enunciado, es decir, el nombre de las clases, atributos y métodos deben ser tal y como se definen en este enunciado.

6.3. Fechas de entrega

Para la convocatoria ordinaria (junio) todos los estudiantes deberán entregar su práctica en el entorno virtual antes del **[fecha de entrega]**, todas las fechas inclusive.

Los tutores deberán evaluar las prácticas y subir las notas al entorno virtual antes del **[fecha de notas]**. La fecha correspondiente a la convocatoria extraordinaria (septiembre) es el **[fecha extraordinaria]** para estudiantes.

6.4. Evaluación de la práctica

Los tutores tienen a su disposición dos tareas en el espacio habilitado del curso virtual donde introducirán la nota del alumno y el número de sesiones a las que ha asistido respectivamente.

Las prácticas tienen carácter **INDIVIDUAL**, para evitar posibles copias todas las prácticas pasarán por un software detector de ejemplares similares. La detección de prácticas copiadas implicará un **SUSPENSO en TODO el curso, es decir, convocatorias de junio y septiembre, para todos los implicados.** Se incluirá la salida de prácticas realizada por IA en el sistema de detección de copias.
Para obtener la máxima nota en la práctica es necesario realizar la parte opcional.
Es requisito indispensable para aprobar el examen la asistencia a las 3 sesiones obligatorias de la práctica, así como superar la propia práctica.
Las prácticas NO se guardan de un curso para otro.

6.5. Preguntas al equipo docente

Las preguntas relativas a la instalación del entorno de desarrollo, puesta en funcionamiento y errores de compilación deben ser remitidas a los tutores de los centros asociados.

El equipo docente atenderá preguntas de carácter metodológico y de diseño, preferentemente a través de los foros del curso virtual de la asignatura y del siguiente correo electrónico:

[correo del equipo docente]

Los datos de contacto de cada uno de los miembros del equipo docente son:

[Nombre y apellidos], *[Cargo]*
Tutorías: [días y horas]
Teléfono: [teléfono]
Mail: [correo]

[Nombre y apellidos], *[Cargo]*
Tutorías: [días y horas]
Teléfono: [teléfono]
Mail: [correo]

[Nombre y apellidos], *[Cargo]*
Tutorías: [días y horas]
Teléfono: [teléfono]
Mail: [correo]